

Registers, Instructions, and the Instruction Cycle

Week 4 — The Machine That Steps Through an Instruction

Dr. Auqib Hamid Lone

Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering

Government College of Engineering & Technology (GCET), Ganderbal

August 7, 2026

Where We Left Off (Week 3)

Unit I was about *data*

- ▶ Numbers: two's complement for integers, IEEE 754 for reals — *how bits represent values*.
- ▶ Arithmetic: adders and the ALU — *how hardware computes on those bits*.

The pivot today

Bits and adders do not run a program by themselves. Something must *fetch* an instruction, *tell* the ALU what to do, and move values between places. This week: the **machine** that steps through an instruction — its storage spaces (registers, stack), its instruction format, and the fetch–decode–execute cycle that animates them.

Roadmap: Four Questions, One Thread

1. **Register organization** — what storage lives *inside* the CPU, and how do the registers talk to each other and the ALU?
2. **Stack organization** — what does it mean to store operands in a LIFO stack, and what hardware implements it?
3. **Instruction formats** — how many addresses does an instruction carry, and what does that choice cost?
4. **Instruction cycle + operand types** — how does the CPU step a single instruction from fetch to execution?

Running thread for today

We will compile $X = (A + B) \times (C + D)$ in *every* instruction format we meet — counting instructions and memory accesses — then trace one instruction through the cycle it runs in.

Today: the storage spaces, and the cycle that steps through an instruction.

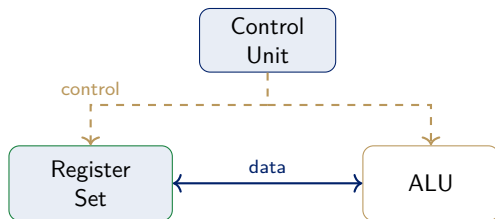
Where Do Operands Live, Anyway?

Socratic question

A program keeps X, A, B, C, D somewhere. Before the ALU can add, *values must move* from where they are stored to the ALU's inputs — and back. How fast can each storage location respond?

- ▶ Main memory is big but slow: a *read* takes many CPU cycles.
- ▶ **Registers** are tiny but fastest — accessible in *one* cycle.
- ▶ Real CPUs keep the operands currently being worked on in registers; the compiler decides what earns a register.

The CPU's Major Components



Three parts do all the work: the **register set** holds the values, the **ALU** computes on them, and the **control unit** (Weeks 6–7) decides what happens when — cf. Mano, *Computer System Architecture*, 3rd ed., Sec. 8-1, p.241 [2].

Registers the Course Knows So Far

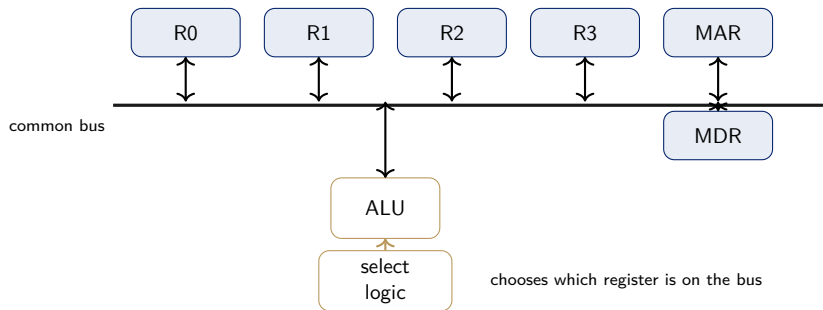
Program-visible registers (Mano, Sec. 8-2, p.242)

- ▶ **General-purpose** register file: $R_0, R_1, \dots, R_{n-1}$ — operands and results; any register can hold an address or a value.
- ▶ **Accumulator** AC: the implicit operand/result register in one-address (accumulator) ISAs.
- ▶ **Stack pointer** SP: holds the address of the stack top — a whole section on that in a moment.

Internal, invisible-to-programmer registers

PC (address of next instruction), IR (current instruction), MAR (address for the current memory access), MDR (data in transit to/from memory). The program never names them — the control unit drives them through the instruction cycle.

How Registers Talk: the Common Bus System



One shared path carries values among registers, ALU, and memory — cf. Mano, Fig. 5-4, p.129 (registers and memory on the common bus system) [2]. A **select** unit (decoder) chooses, each cycle, *which* register drives the bus.

Register-Selection Signals: SELA / SELB / SELD

How a transfer picks its source and destination (Mano, Table 8-1, p.243)

- ▶ SELA — which register's contents go to the ALU's *first* input.
- ▶ SELB — which register's contents go to the ALU's *second* input.
- ▶ SELD — *which* register is loaded with the result.

The idea

Register selection is just *encoded* in a few select bits and decoded into one-hot “drive bus” / “load now” signals. That is the whole trick behind moving values between registers without hard-wiring every pair.

Worked Example: $R1 \leftarrow [R2]$ over the Bus

Transfer request

Copy the *contents* of R2 into R1 (notation: the contents of a register X are written $[X]$).

1. SELA encodes “R2” — R2’s output enable goes high.
2. R2’s contents are driven onto the common bus.
3. SELD encodes “R1” — R1’s load-enable goes high.
4. On the clock edge, R1 captures whatever the bus holds: $R1 \leftarrow [R2]$.

No direct $R2 \rightarrow R1$ wire exists — every transfer uses the bus plus two select signals.
Same mechanism drives every register move and ALU operand.

Socratic Check: Why Not Put *Everything* in Registers?

Question

If registers are the fastest storage, why does a program put X, A, B, C, D in memory instead of keeping everything in $R0 \dots R_{n-1}$?

- ▶ **Registers are scarce**: a few dozen at most — the program cannot keep megabytes of data there.
- ▶ **Cost**: each register costs silicon area and consumes power; large register files are expensive.
- ▶ **Registers are a *scratchpad* for what is *being worked on***: values rest in memory (big, cheap, persistent) and are loaded into registers only while the ALU touches them.

Registers are the fast scratchpad. A stack is a different way to organize operands.

Motivation: How Do You Evaluate a Postfix Expression?

Socratic question

Humans write $A + B$; machines with one ALU must feed it two operands. How would a calculator evaluate $A B +$ if it could only look at the *top* item of a scratch list — no registers, no random access?

- ▶ The rule: read a value, **push** it; read an operator, **pop** the last two values, compute, **push** the result.
- ▶ The *last* value pushed is the *first* one taken back out — **last-in, first-out** (LIFO). That is a **stack**.
- ▶ Many machines (HP calculators, the JVM) use exactly this model.

Definition: Stack, SP, PUSH/POP

Stack

A **stack** is an ordered storage structure where all insertions and deletions happen at *one end*, the **top**, addressed by a dedicated register SP (the **stack pointer**). Behavior is LIFO (Mano, Sec. 8-3, p.247) [2].

- ▶ **PUSH**: $SP \leftarrow [SP] - 1$, then $M[SP] \leftarrow \text{operand}$ — store at the new top. (Stack grows *downward* toward lower addresses.)
- ▶ **POP**: $\text{operand} \leftarrow M[SP]$, then $SP \leftarrow [SP] + 1$ — remove the current top.

Worked micro-transfer

Stack contains $[?, 5, 7]$, top at 7. PUSH 9 $\Rightarrow$ stack $[?, 5, 7, 9]$; POP $\Rightarrow$ returns 9, stack back to $[?, 5, 7]$.

Worked Example: $A + B$ on a Stack Machine

Step	Operation	Stack after (top right)	Effect
1	PUSH A	A	value A stored at top
2	PUSH B	B A	value B stored above A
3	ADD	A+B	pop B, pop A, push result
4	POP X	<i>empty</i>	store top into X

Notice

The ADD instruction names *no operands at all* — both come implicitly from the top of the stack. This is the **zero-address** instruction format we meet in Act 3.

Register Stack vs. Memory Stack

Two hardware homes for a stack (Mano, Sec. 8-3, p.248)

- ▶ **Register stack:** the stack lives inside the CPU in a set of registers — very fast, but only a handful of words (e.g. 64 words).
- ▶ **Memory stack:** the stack is a region of main memory, with SP holding the address of the current top — as large as memory, but every PUSH/POP is a memory access.

Why real machines choose memory stacks

Function calls nest deeply (recursion); a few registers cannot hold the frames. Real ISAs keep SP in a register and store stack frames in memory — registers are the pointer, memory is the stack.

Socratic Check: Registers vs. Stack as the Operand Home

Question

Both a register file and a stack can supply the ALU's operands. What does a stack force you to do that registers do not?

- ▶ **Access is restricted**: you can only reach the *top* — no random access to $R0 \dots Rn-1$ -style slots mid-computation.
- ▶ **Reordering costs**: to use a value that is buried, you must pop everything above it first (or shuffle).
- ▶ **But instructions shrink**: operands are implicit — hence the compact **zero-address** format. Compilers love the compactness; hardware pays with restricted access.

The operand's home (registers, memory, stack) shapes what an instruction must say.

Motivation: What Must an Instruction Say?

Socratic question

The CPU must be told *what* to do and *where* the operands are. How much “where” information must a machine instruction physically carry?

- ▶ Minimum: an **opcode** — which operation (ADD, LOAD, ...).
- ▶ Beyond that: *addresses* of operands and results — and each address costs bits in the instruction word.
- ▶ The trade-off is fundamental: **more addresses per instruction** $\Rightarrow$ more work done per instruction but **longer instructions**; fewer addresses $\Rightarrow$ compact but **more instructions** (and the hardware must know where the operands are implicitly).

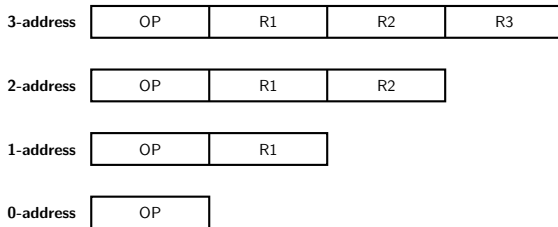
Definition: Instruction Format

Instruction format

An **instruction format** is the layout of an instruction word: an **opcode** field naming the operation, followed by zero or more **address/operand** fields naming where operands and results live (Mano, Sec. 8-4, p.255) [2].

- ▶ Fields are fixed-width bits; the opcode width sets an upper bound on how many distinct instructions the ISA can name.
- ▶ Address fields can name registers (short, cheap) or memory locations (long, flexible) — Week 5 turns those fields into real addresses via **addressing modes**.

The Four Format Families



As the operand count falls, instructions get shorter but the machine must find operands *implicitly* (accumulator or stack top).

Three- and Two-Address Forms

3-address: $\text{ADD } R1, R2, R3 \Rightarrow R1 \leftarrow [R2] + [R3]$

Operands and destination are all explicit — the most flexible, longest form.

2-address: $\text{ADD } R1, R2 \Rightarrow R1 \leftarrow [R1] + [R2]$

The first field doubles as one operand *and* the result destination (**destructive** — the old value of R1 is lost).

Fewer fields $\Rightarrow$ shorter instructions and smaller programs, at the cost of destroying one operand and needing explicit copy instructions.

One- and Zero-Address Forms

1-address (accumulator): $\text{ADD } A \Rightarrow AC \leftarrow [AC] + [A]$

One operand is always the **accumulator** AC, named by no field; the single field names the other operand (usually memory).

0-address (stack): $\text{ADD} \Rightarrow \text{pop two, push sum}$

Both operands come from the stack top — the format carries *only* the opcode; every operand is implicit.

Extreme compactness, but every non-stack access needs explicit PUSH/POP instructions.

Worked Example: $X = (A+B) \times (C+D)$ — 3-Address

Three-address: 3 instructions, operands all named

- ▶ `ADD R1, A, B` $\Rightarrow R1 \leftarrow A + B$
- ▶ `ADD R2, C, D` $\Rightarrow R2 \leftarrow C + D$
- ▶ `MUL X, R1, R2` $\Rightarrow X \leftarrow R1 \times R2$

Tally

3 instructions; each carries three addresses — wide words, but the program is short and needs no extra data movement.

Worked Example: Same Expression, 2-Address

Two-address: 5 instructions — destructive form needs copies

- ▶ `MOV R1, A` $R1 \leftarrow A$ (explicit copy, since `ADD` overwrites)
- ▶ `ADD R1, B` $R1 \leftarrow R1 + B$
- ▶ `MOV R2, C` $R2 \leftarrow C$
- ▶ `ADD R2, D` $R2 \leftarrow R2 + D$
- ▶ `MUL X, R1` $X \leftarrow R1 \times R2$ ($R2$ is the implicit second operand)

Tally

5 instructions vs. 3 — the 2-address format is cheaper per instruction but needs **extra copy instructions** because `ADD` is destructive.

Worked Example: Same Expression, 1- and 0-Address

One-address (accumulator): 7 instructions, operands via AC + memory

LOAD A; ADD B; STORE T; LOAD C; ADD D; MUL T; STORE X — with a temporary T in memory because the accumulator cannot hold two partial results at once.

Zero-address (stack): 8 instructions, operands implicit

PUSH A; PUSH B; ADD; PUSH C; PUSH D; ADD; MUL; POP X — shortest instructions, but the most of them.

Same expression, four formats: 3, 5, 7, and 8 instructions. The format choice is a real, countable trade-off.

Instruction Formats: Side by Side

Format	Example	Program size for $X = (A+B) \times (C+D)$	Operand cess	ac-	Typical use
3-address	ADD R1,R2,R3	3 instr., wide words	all explicit		register ISAs (RISC)
2-address	ADD R1,R2	5 instr. + copies	one destroyed		older CISC
1-address	ADD A	7 instr., needs temps	accumulator		simple CPUs
0-address	ADD	8 instr., compact	stack top		stack machines, JVM

More addresses per instruction $\Rightarrow$ fewer instructions but wider words; fewer addresses $\Rightarrow$ more instructions but compact.

We have the storage spaces and the formats. Now: how does the CPU step one instruction?

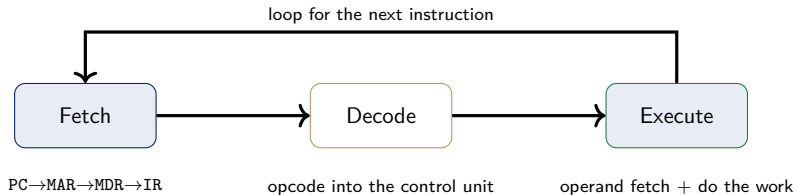
Motivation: Who Runs the Show, Step by Step?

Socratic question

ADD R1, A sits in memory. For it to take effect, the CPU must bring the instruction in, understand it, collect [R1] and [A], add, and store back. What *sequence of basic steps* does the machine repeat for *every* instruction?

- ▶ The answer is a fixed skeleton — the **instruction cycle** — that the control unit runs over and over, once per instruction.
- ▶ The skeleton is the *same* for every instruction; only the **execute** phase differs by opcode.

The Instruction Cycle



Fetch → decode → execute is the standard instruction-cycle skeleton (Hamacher, *Computer Organization*, Ch. 7; standard treatment) [1].

The Fetch Phase, Step by Step

Fetch transfers (PC knows where the next instruction lives)

- ▶ $MAR \leftarrow [PC]$ — remember the address.
- ▶ Initiate a memory *read* at MAR.
- ▶ $MDR \leftarrow \text{memory data}$ — the instruction word comes back into the CPU.
- ▶ $IR \leftarrow [MDR]$ — the instruction now sits in the instruction register.
- ▶ $PC \leftarrow [PC] + 1$ — prepare the *next* fetch.

Why these registers?

PC holds the next address, MAR pins it during the access, MDR buffers the incoming word, IR holds the current instruction while it is decoded. Each register exists for *one* reason in the cycle.

The Decode and Execute Phases

Decode

The opcode bits of IR tell the control unit *which* operation this is — the unit then selects the right sequence of control signals (exactly what we build in Weeks 6–7).

Execute — the operand-dependent part

- ▶ If a memory operand is needed, its address is resolved (addressing modes, Week 5) and the operand fetched into a register.
- ▶ The ALU performs the operation; the result is written to the destination named in the instruction.
- ▶ The *number* of execute steps varies by instruction — fetch is fixed, execute is not.

Worked Example: One Full Cycle for ADD R1, A

Phase	Register activity	Memory / ALU	State
Fetch	$MAR \leftarrow [PC]$; $MDR \leftarrow instr.$; $IR \leftarrow [MDR]$; $PC \leftarrow [PC] + 1$	read instruction word	instruction in IR
Decode	IR opcode $\rightarrow$ control unit	—	operation identified
Execute	operand [A] fetched; $AC \leftarrow [AC] + [A]$	read operand; ALU adds	result in AC

Takeaway

The same three phases run for every instruction — only the execute row changes. This is the loop the CPU is always inside.

Types of Operands

What the fields of an instruction can name (Hamacher, Ch. 7; standard treatment) []

- ▶ **Addresses**: a location in memory (used by LOAD, STORE, BRANCH).
- ▶ **Numbers**: integer or floating-point values the ALU computes on.
- ▶ **Characters**: text — usually ASCII/Unicode codes stored in a byte or halfword.
- ▶ **Logical data**: bit patterns — each bit is its own value; operations like AND, OR, XOR, shift work bitwise.

Why this matters

The *type* determines how the ALU wires the bits: a branch needs the address, an integer add needs the value, a shift needs the bit mask. The opcode says which interpretation applies.

Socratic Check: Tying the Week Together

Question

For ADD R1, A on an accumulator machine: which register holds the next instruction's address, which holds the current instruction, and where do the two operands of the ADD come from?

- ▶ PC points at the next instruction; IR holds the current one after fetch.
- ▶ First operand: AC (accumulator — the implicit operand of the 1-address format).
- ▶ Second operand: memory location A, fetched during execute.
- ▶ Every piece of this answer came from Acts 1–4 — the registers, the formats, and the cycle are one connected machine.

One instruction, one cycle: registers hold, formats name, the cycle runs.

The Three Threads Meet

Storage spaces and formats are two sides of one choice

- ▶ **Registers**: fast, explicit, random access — buys the wide 3-address format.
- ▶ **Stack**: compact, restricted, implicit — buys the 0-address format.
- ▶ The instruction cycle is the *glue*: it steps whichever format the ISA chose, through the same fetch/decode/execute skeleton.

The running thread, closed

$X = (A + B) \times (C + D)$ ran in 3, 5, 7, or 8 instructions depending on the format — and each of those instructions then lives through one full fetch–decode–execute cycle.

Bridge to Week 5

What we built today

- ▶ The CPU's storage: registers (fast scratchpad), stack (LIFO operand home), memory (big and slow).
- ▶ Four instruction formats, with a counted cost per format.
- ▶ The fetch–decode–execute cycle and the four types of operands.

What's still missing

We wrote “ADD R1, A” and said “operand [A] is fetched” — but we never said **how** the field A turns into the actual location of the operand. That is **addressing modes**, next week — together with the datapath (buses) that actually moves the values.

Summary

- ▶ **Register organization:** registers are the one-cycle scratchpad; transfers happen over a common bus selected by SELA/SELB/SELD.
- ▶ **Stack organization:** SP addresses a LIFO top; PUSH/POP with ± 1 on SP; register stacks are fast, memory stacks are large.
- ▶ **Instruction formats:** 3-, 2-, 1-, 0-address — the address count trades instruction width against program size (3 vs. 5 vs. 7 vs. 8 instructions for one expression).
- ▶ **Instruction cycle:** fetch (PC $\rightarrow$ MAR $\rightarrow$ MDR $\rightarrow$ IR, PC+1), decode, execute.
- ▶ **Operand types:** addresses, numbers, characters, logical data — the opcode picks the interpretation.

References



V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky.

Computer Organization.

McGraw-Hill, 5th edition, 2002.



M. Morris Mano.

Computer System Architecture.

Prentice-Hall, 3rd edition, 1993.